COMPLETE BEGINNER GUIDE

Jenkins

JENKINS

FOR BEGINNERS

HANDBOOK

- STEP-BY-STEP EXPLANATIONS
- HANDS-ON EXAMPLES
- PIPELINES MADE EASY
- CI/CD FUNDAMENTALS
- PRACTICAL PROJECT

FROM ZERO TO YOUR FIRST CI/CD PIPELINE

BUILD | TEST | DEPLOY | MONITOR | SECURE | AUTOMATE

VERIQTA | LEARN • PRACTICE • AUTOMATE • SUCCEED

YOUTUBE GITHUB LINKEDIN X INSTAGRAM TELEGRAM

JENKINS HANDBOOK FOR BEGINNERS

1. JENKINS FROM ZERO

WHAT JENKINS IS

Jenkins is an open-source automation server that helps you automate the building, testing, and deployment of your software.
It supports Continuous Integration and continuous Delivery.

WHY JENKINS EXISTS

To help teams deliver software faster, more reliably, and with fewer manual errors by automating repetitive tasks.

THE PROBLEM JENKINS SOLVES

- Manual builds are slow and inconsistent.
- Human errors during build and deployment.
- Hard to track what changed and when.
- Difficult to repeat the same process reliably.
- Delays in releasing new features or fixes.

MANUAL SOFTWARE DELIVERY VERSUS AUTOMATION

MANUAL DELIVERY		AUTOMATED DELIVERY
Slow	VS	Fast
Error-prone		Reliable
Inconsistent		Consistent
Hard to scale		Scalable
Not repeatable		Repeatable

WHERE JENKINS FITS INTO DEVOPS

Jenkins is the automation engine in DevOps.
It bridges the gap between Development and Operations by automating the build, test, and delivery process.

CI VERSUS CD

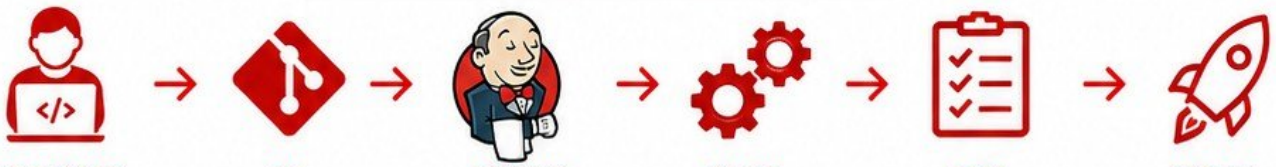
CONTINUOUS INTEGRATION (CI)

- Developers integrate code frequently.
- Jenkins builds and tests every change.
- Goal: Find and fix issues early.

CONTINUOUS DELIVERY (CD)

- Code that passes all tests is automatically prepared for release.
- Goal: Deliver working software to users quickly and safely.

SIMPLE JENKINS WORKFLOW



DEVELOPER
Writes code and pushes changes to Git repository.

GIT
Stores the code and version history.

JENKINS
Detects changes and starts the automation process.

BUILD
Compiles the code and prepares the application.

TEST
Runs automated tests to verify the application.

DEPLOY
Deploys the application to the target environment.

JENKINS HANDBOOK FOR BEGINNERS

JENKINS ARCHITECTURE

CORE JENKINS COMPONENTS

JENKINS CONTROLLER
The brain of Jenkins. It manages jobs, schedules builds, stores configuration, and coordinates agents.

JENKINS AGENTS
Machines that run the actual builds. Agents can be on the same machine or on different servers (Windows, Linux, etc.).

JOBS
A job is a set of instructions that tells Jenkins what to do: build, test, package, and more.

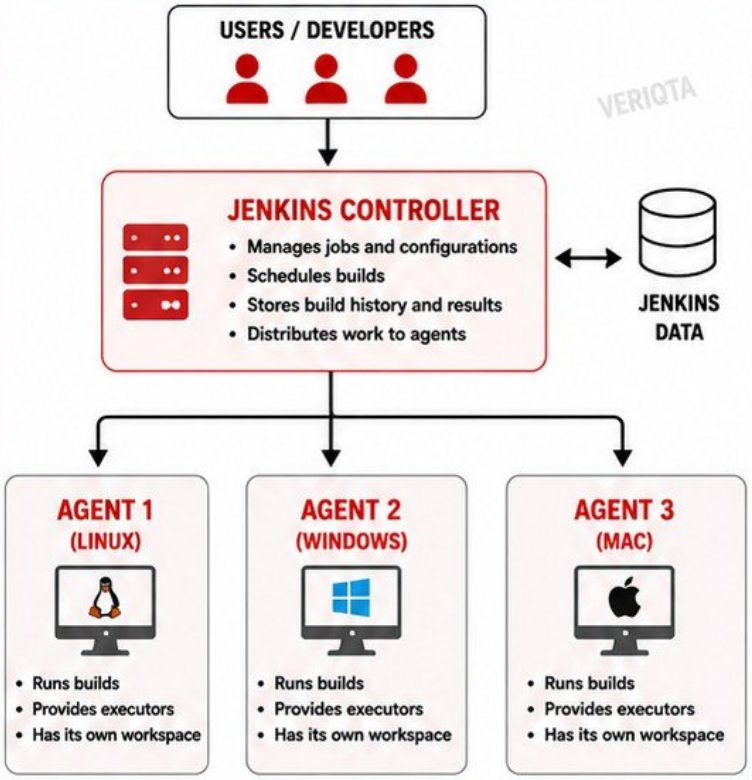
BUILDS
A build is a single execution of a job. Every build has a unique number and a result (Success/Fail).

EXECUTORS
Slots on the controller or agent that execute builds. One executor can run one build at a time.

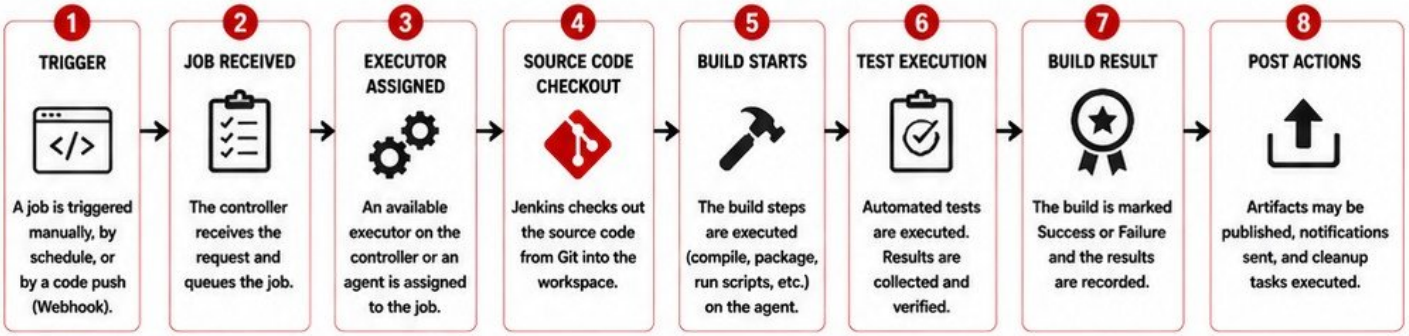
WORKSPACES
A directory on the agent where the source code is checked out and the build runs.

PLUGINS
Extend Jenkins features and add new integrations (Git, Docker, Slack, AWS, and many more).

SIMPLE JENKINS ARCHITECTURE (CONTROLLER → AGENTS)



HOW A JENKINS JOB MOVES THROUGH THE SYSTEM



KEY TAKEAWAYS

- The Controller manages everything.
- Agents do the actual work.
- Jobs define what Jenkins must do.
- Executors run builds.
- Workspaces store code while building.
- Plugins extend Jenkins capabilities.

ARCHITECTURE IN ONE LINE

Jenkins Controller plans and manages work; Jenkins Agents execute the work.

TIP

You can run Jenkins with no agents (everything on the controller) or with many agents for better performance and isolation.

JENKINS HANDBOOK FOR BEGINNERS

3. INSTALLING JENKINS

1 BASIC SYSTEM REQUIREMENTS

- 2 GB RAM minimum (4 GB recommended)
- 50 GB disk space minimum
- 64-bit OS
- Internet connection
- Java 17 or Java 11

2 JAVA REQUIREMENT

Jenkins requires Java to run.

Check Java installation:

```
java -version
```

Install Java (Example for Ubuntu):

```
sudo apt update
sudo apt install openjdk-17-jdk -y
```

3 INSTALLING JENKINS (UBUNTU EXAMPLE)

1. ADD JENKINS REPOSITORY

```
curl -fsSL https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key | sudo tee /usr/share/keyrings/jenkins-keyring.asc > /dev/null
```

2. ADD REPOSITORY TO SOURCES LIST

```
echo "deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] https://pkg.jenkins.io/debian-stable binary/" | sudo tee /etc/apt/sources.list.d/jenkins.list > /dev/null
```

3. UPDATE AND INSTALL JENKINS

```
sudo apt update
sudo apt install jenkins -y
```

4. VERIFY INSTALLATION

```
jenkins --version
```

4 STARTING AND STOPPING JENKINS

Start Jenkins

```
sudo systemctl start jenkins
```

Stop Jenkins

```
sudo systemctl stop jenkins
```

Restart Jenkins

```
sudo systemctl restart jenkins
```

Check Status

```
sudo systemctl status jenkins
```

5 DEFAULT JENKINS PORT

Jenkins runs on port:

8080

URL: http://your-server-ip:8080

6 ACCESSING JENKINS

Open your browser and go to:

http://your-server-ip:8080

Replace your-server-ip with your actual server IP address or domain name.

7 INITIAL ADMINISTRATOR PASSWORD

On the Unlock Jenkins page, you need the initial password. It is located at:

```
/var/lib/jenkins/secrets/initialAdminPassword
```

View the password:

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

Copy the password and paste it on the Unlock Jenkins page.

8 UNLOCK JENKINS SCREEN

9 FIRST-TIME SETUP

1. Choose "Install suggested plugins"
2. Jenkins installs the required plugins.
3. Create the first admin user.
4. Configure instance settings (Jenkins URL).
5. Complete the setup.
6. Jenkins is ready to use!

4. JENKINS DASHBOARD AND INTERFACE

1. NEW ITEM
Create a new job. Freestyle project, Pipeline, or Multi-configuration project.

2. BUILD HISTORY
View recent builds of your jobs. Check build status and results.

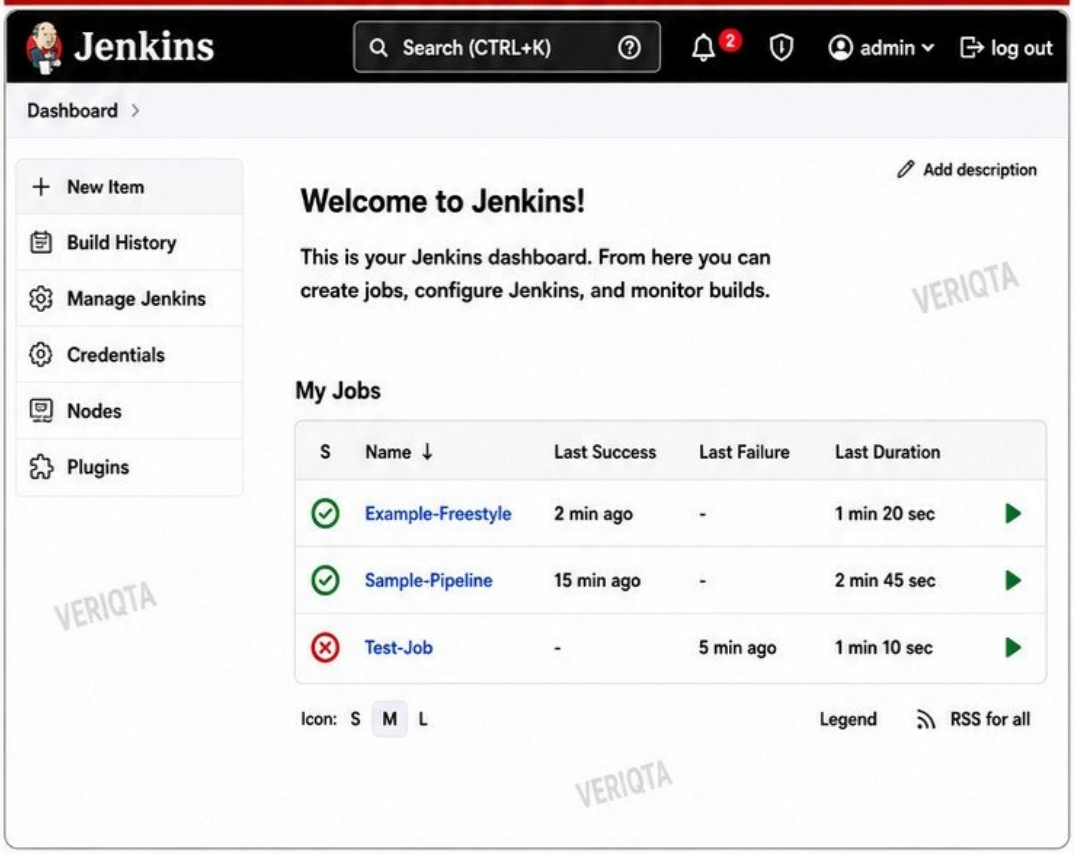
3. MANAGE JENKINS
Configure global settings. Manage system configuration.

4. CREDENTIALS
Store usernames, passwords, SSH keys, and secret data securely.

5. NODES
Manage agents/nodes that run builds. Add, configure or monitor nodes.

6. PLUGINS
Install, update or remove plugins to extend Jenkins functionality.

JENKINS DASHBOARD OVERVIEW



7. JOB PAGES
Click a job name to open its page. Here you configure, run, and view everything about the job.

8. BUILD NUMBERS
Each build gets a unique number. Example: #1, #2, #3
Useful for tracking changes over time.

9. CONSOLE OUTPUT
Shows detailed logs of everything that happened during the build.
Key place to find errors and debug.

10. IMPORTANT SETTINGS
Beginners can find important settings in these places:
- Manage Jenkins
- Global Tool Configuration
- Configure System
- Manage Nodes
- Configure Credentials

WHERE BEGINNERS CAN FIND IMPORTANT SETTINGS

OPEN JENKINS DASHBOARD
Go to http://your-server:8080

→ **MANAGE JENKINS**
Global settings, tools, system configuration.

→ **MANAGE NODES**
Add and configure agents to run builds.

→ **MANAGE CREDENTIALS**
Add and manage usernames, passwords, keys.

→ **MANAGE PLUGINS**
Install or update plugins to add new features.

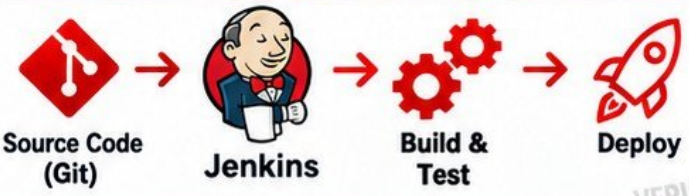
TIP FOR BEGINNERS: The Console Output is your best friend when something goes wrong. Always check it first!

5. YOUR FIRST JENKINS JOB

WHAT A JENKINS JOB IS

A **Jenkins job** is a task or project in Jenkins that runs a build, test and deployment process.

JENKINS JOB WORKFLOW



CREATING A FREESTYLE PROJECT

1. Click **New Item** on the Jenkins dashboard.
2. Enter a job name.
3. Select **Freestyle project.**
4. Click **OK.**
5. Configure the job (source code, build steps, etc.).
6. Click **Save.**

NAMING THE JOB

Use a clear and simple name, for example: **my-app-build.**

ADDING A DESCRIPTION

Add a short description so you know what the job does.

BUILD STEPS – EXECUTE SHELL

- Scroll to **Build Steps.**
- Choose **Execute shell.**
- Add a simple command, for example:

```
echo "Hello Jenkins"
```

SAVING THE CONFIGURATION

Click **Save** when you finish configuring the job.

BUILD NOW

Click **Build Now** to start a build.

Jenkins will run the job and show the progress.

VIEWING THE FIRST BUILD

- Go to **Build History.**
- Click the latest build number.
- View **Console Output.**
- Check that the build is SUCCESS.

SUCCESS VERSUS FAILURE

SUCCESS

- Build is green.
- All steps ran successfully.
- Code is ready for next steps.

FAILURE

- Build is red.
- One or more steps failed.
- Check **Console Output** for the error.

JENKINS HANDBOOK FOR BEGINNERS

6. UNDERSTANDING JENKINS BUILDS

1. WHAT HAPPENS WHEN JENKINS STARTS A BUILD

BUILD TRIGGERED
You click Build Now or a trigger starts the build.

→

JENKINS QUEUES THE JOB
Jenkins adds the job to the queue.

→

EXECUTOR PICKS UP JOB
An available executor starts running the build.

→

STEPS ARE EXECUTED
Each build step runs in order (checkout, build, test, etc.).

→

BUILD COMPLETES
Jenkins records the result and shows the build status.

2. BUILD NUMBERS

Every build gets a unique number.

Example:
#1, #2, #3 ...

Build numbers help you track changes over time.

3. BUILD STATUS

SUCCESS — Build completed with no errors.

FAILURE — One or more steps failed.

UNSTABLE — Build completed but tests failed.

4. BUILD HISTORY

Shows a list of recent builds for a job.

S	#	DATE	STATUS	DURATION
✓	#5	May 21, 2024 10:30 AM	✓	45 sec
✗	#4	May 21, 2024 10:20 AM	✗	1 min 12 sec
○	#3	May 21, 2024 10:10 AM	↻	50 sec
✓	#2	May 21, 2024 10:00 AM	✓	40 sec
✓	#1	May 21, 2024 9:50 AM	✓	38 sec

5. CONSOLE OUTPUT

Shows detailed logs of everything that happened during the build.

Best place to find errors and debug problems.

6. WORKSPACE

A workspace is a folder on the agent where Jenkins checks out the code and runs the build.

Example location:

```
/var/lib/jenkins/workspace/MyJob
```

Files created during the build live here.

7. BUILD DURATION

Jenkins shows how long each build took.

Helps you monitor performance and identify slow builds.

8. SUCCESSFUL BUILDS

All steps passed.
No errors.
Artifacts may be created.
Application is ready to use or deploy.

9. FAILED BUILDS

One or more steps failed.

Build stops at the failed step.

Check the Console Output to see what went wrong.

Fix the issue and build again.

10. READING SIMPLE JENKINS ERRORS

Example Error Messages:	How to Fix:
`ERROR: command not found` → The required tool is not installed or not in PATH.	• Install the missing tool or configure PATH.
`Permission denied` → Jenkins user does not have permission to run the command or access the file.	• Fix file permissions or run with correct user.
`Failed to connect to repository` → Check network, URL, or credentials.	• Check credentials, repository URL, and network.

- Build = A single run of a job.
- Each build has a number and a status.
- Use Build History to review past builds.
- Console Output helps you understand what happened.
- Workspace contains the files used in the build.
- Always read errors carefully and fix the root cause.

JENKINS HANDBOOK FOR BEGINNERS

7. CONNECTING JENKINS TO GITHUB

1 WHY JENKINS NEEDS SOURCE CODE

Jenkins needs your source code to build, test and package your application.

2 GIT BASICS REQUIRED FOR JENKINS

- Git is a version control system.
- Code is stored in repositories.
- Jenkins pulls (clones) code from the repository.
- You need: Repository URL and access credentials.

3 REPOSITORY URL

Copy the HTTPS or SSH URL of your GitHub repository.

EXAMPLE (HTTPS)

```
https://github.com/username/my-repo.git
```

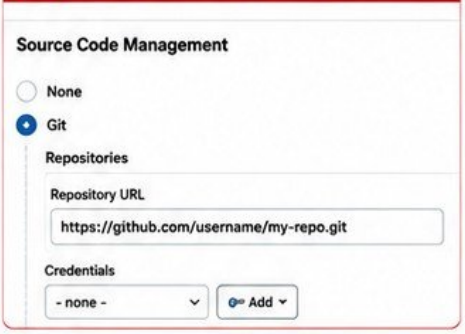
EXAMPLE (SSH)

```
git@github.com:username/my-repo.git
```

4 INSTALLING GIT SUPPORT

- Jenkins needs Git installed on the server (or agent).
- Go to Manage Jenkins > Tools.
- Add Git installation or use default if already available.
- Click Save.

5 ADDING A GIT REPOSITORY



6 SELECTING A BRANCH

Choose the branch Jenkins should build.

Branch Specifier (blank for 'any')

```
*/main
```

Common branches:
main, master, develop

7 CREDENTIALS FOR PRIVATE REPOSITORIES

- Private repositories require credentials.
- Go to Manage Jenkins > Credentials.
- Add: Username with password or SSH Username with private key.
- Select the credential in the job configuration.

8 TESTING REPOSITORY ACCESS

- Save the job configuration.
- Click "Test Connection" (if available).
- Or build the job.
- Jenkins will try to access the repository and show the result.

9 JENKINS CLONING WORKFLOW

GITHUB REPOSITORY	JENKINS CONNECTS	CLONE REPOSITORY	BUILD STARTS
Code is stored in GitHub.	Jenkins connects using URL and credentials.	Jenkins clones the code to the workspace.	Jenkins runs the build using the cloned code.

QUICK STEPS SUMMARY

TIP: Start with a public repository first to make sure everything works. Once Jenkins can clone the code successfully, you can move on to private repositories.

8. JENKINS CREDENTIALS

1 WHY CREDENTIALS SHOULD NOT BE PLACED DIRECTLY IN JOBS

- Plain text can be read by anyone who can access the job.
- It may be seen in build logs.
- It may be exposed in configuration files or scripts.
- Credentials in jobs are hard to manage and reuse.

USE THE JENKINS CREDENTIALS STORE TO KEEP SECRETS SAFE AND REUSABLE.

2 TYPES OF CREDENTIALS

USERNAME AND PASSWORD	Stores a username and password. Used for APIs, servers, databases, and other services.
SECRET TEXT	Stores a secret string. Used for tokens, API keys, and other secret values.
SSH KEYS	Stores SSH private keys. Used for secure Git access and server authentication.
SECRET FILE	Stores secret files such as certificates, JSON files, or configuration files.

3 CREDENTIAL IDs

- Every credential has a unique ID.
- ID is used in jobs and pipelines to reference the credential.
- Use meaningful and consistent IDs.

Example: github-token, docker-cred, prod-ssh-key

4 CREDENTIALS SCOPES

- **Global:** Available to all jobs in Jenkins.
- **Folder:** Available only to jobs in a specific folder.
- **System:** Available for system use.

Prefer the most restricted scope possible.
Less access = More security.

5 ADDING CREDENTIALS (STEP-BY-STEP)

1. Go to Manage Jenkins > Credentials
2. Click Add Credentials
3. Choose the credential type
4. Enter the details (username, password, secret, or SSH key)
5. Select the scope
6. Click OK / Save

6 SELECTING CREDENTIALS INSIDE A JOB

You can use credentials in build steps, environment variables, or pipeline.

Examples:

- Username and password in form fields
- Using credentials() in Pipeline
- Using withCredentials() step

PIPELINE EXAMPLE

```
withCredentials([usernamePassword(
 credentialsId: 'github-cred',
 usernameVariable: 'GIT_USER',
 passwordVariable: 'GIT_PASS')]) {
   sh 'git push https://$GIT_USER:
   $GIT_PASS@github.com/repo.git'
}
```

7 BASIC CREDENTIAL SAFETY

- Never hardcode secrets in scripts, Jenkinsfiles, or configuration files.
- Use the least privileged credentials possible.
- Rotate secrets regularly.
- Limit who can view and use credentials.
- Audit access to sensitive credentials.

JENKINS HANDBOOK FOR BEGINNERS

9. BUILD TRIGGERS

BUILD TRIGGERS TELL JENKINS WHEN TO START A BUILD.

1 MANUAL BUILDS

- You start the build manually from the Jenkins UI by clicking "Build Now".
- Useful for testing changes or running builds on demand.

2 POLL SCM

- Jenkins checks the source code repository at regular intervals for changes.
- If a change is detected, Jenkins starts a build.

3 SCHEDULED BUILDS

- Builds run automatically at a specific date and time using a schedule (cron).
- Useful for nightly builds, reports, or regular tasks.

4 WEBHOOKS

- An external system (like GitHub) notifies Jenkins immediately when an event happens.
- Enables real-time builds without waiting.

5 TRIGGERING JENKINS AFTER A GIT PUSH

1. Developer pushes code to GitHub.
2. GitHub sends a webhook event.
3. Jenkins receives the webhook.
4. Jenkins automatically starts the build.
5. Build runs and you get results.

WEBHOOKS ARE THE FASTEST AND MOST EFFICIENT WAY TO TRIGGER BUILDS AFTER CODE CHANGES.

6 BASIC CRON SYNTAX (FOR SCHEDULED BUILDS)

FIELD	MEANING
*	Any value
H	Hash value (spread load)
M	Minutes (0 – 59)
H	Hours (0 – 23)
D	Day of Month (1 – 31)
M	Month (1 – 12)
W	Day of Week (0 – 7)

EXAMPLES

`H/15 * * * *`	Every 15 minutes
`H 2 * * *`	Every day at 2 AM
`0 0 * * 1-5`	Every weekday at midnight
`0 H/2 * * *`	Every 2 hours
`0 30 14 * *`	Every day at 2:30 PM

7 WHEN EACH TRIGGER IS USEFUL

TRIGGER	BEST USED FOR
MANUAL BUILDS	Testing, debug, on-demand builds
POLL SCM	Environments without webhooks, legacy systems
SCHEDULED BUILDS	Nightly builds, reports, database backups, maintenance tasks
WEBHOOKS	CI/CD pipelines, real-time builds after every code push

8 GITHUB → WEBHOOK → JENKINS → BUILD FLOW

GITHUB
Developer pushes code to the repository.

WEBHOOK
GitHub sends a webhook event to the Jenkins URL.

JENKINS
Jenkins receives the webhook and triggers the job.

BUILD STARTS
Jenkins checks out the code and starts the build process.

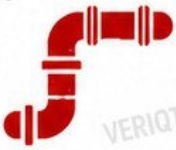
BUILD COMPLETE
Build finishes and results are shown in Jenkins (Success / Failure).

10. INTRODUCTION TO JENKINS PIPELINE

1 WHAT IS A PIPELINE?

A Pipeline is a set of automated steps that define the full delivery process of your software.

It allows you to build, test, and deploy your code in a reliable and repeatable way.



PIPELINE = AUTOMATION + REPEATABILITY + VISIBILITY + RELIABILITY

2 FREESTYLE JOBS VERSUS PIPELINES

FREESTYLE JOBS

- Configured through the UI (many pages).
- Harder to version control.
- Difficult to reuse and share.
- Not easy to see the full flow.
- Good for small and simple tasks.

VS

PIPELINES

- Defined as code (Jenkinsfile).
- Stored in source control.
- Easy to reuse and share.
- Clear stages and flow.
- Best for modern CI/CD workflows.

3 PIPELINE STAGES

Stages represent major phases of your delivery process. Each stage runs in order.

BUILD → TEST → DEPLOY → VERIFY

Stages help organize work, make pipelines readable, and show progress.

4 PIPELINE STEPS

Steps are individual tasks inside a stage.

Examples of steps:

- Checkout code
- Run shell commands
- Compile code
- Run tests
- Deploy applications
- Send notifications

5 JENKINSFILE

A Jenkinsfile is a text file that defines your Pipeline as code.

It lives in your source code repository.

When Jenkins runs, it reads the Jenkinsfile and executes the instructions.

Jenkinsfile

6 DECLARATIVE PIPELINE

A structured and easy way to define pipelines using a fixed syntax.

Uses keywords like: pipeline, agent, stages, steps, post.

Simple, readable, and beginner-friendly.

Most common choice for beginners.

7 SCRIPTED PIPELINE (INTRODUCTION)

A more flexible way to define pipelines using Groovy code.

You have full control with loops, conditions, functions, and more.

More powerful but also more complex.

Best for advanced users and custom logic.

8 WHY BEGINNERS SHOULD START WITH DECLARATIVE PIPELINE

EASY TO READ
Simple syntax that is easy to understand.

STRUCTURED
Built-in structure helps you stay organized.

LESS ERROR-PRONE
Jenkins handles many details for you.

STANDARD SYNTAX
Uses a standard format across the industry.

EASY TO LEARN
Perfect for beginners in CI/CD.

GROW AND SCALE
You can move to Scripted Pipeline when needed.

JENKINS HANDBOOK FOR BEGINNERS

11. YOUR FIRST JENKINSFILE

1 JENKINSFILE KEYWORDS

These are the most important building blocks of a Declarative Pipeline.

PIPELINE	Defines the start of the Pipeline.
AGENT	Defines where the Pipeline runs.
STAGES	A container for all the stages.
STAGE	A single phase of the Pipeline.
STEPS	The tasks executed in a stage.
ECHO	Prints a message in the console.

2 CREATING A SIMPLE JENKINSFILE

Create a file named **Jenkinsfile** in the root of your repository.

```
1  pipeline {
2    agent any
3
4    stages {
5      stage('Hello') {
6        steps {
7          echo 'Hello, Jenkins!'
8        }
9      }
10   }
11 }
13
```

EXPLANATION	
PIPELINE	Starts the pipeline.
AGENT	Runs on any available agent.
STAGES	Container for all stages.
STAGE	A stage named 'Hello'.
STEPS	Tasks inside the stage.
ECHO	Prints a message.

3 SAVING IT INSIDE GIT

- Add the Jenkinsfile to your repository.
- Commit the file.
- Push the changes to your remote Git repository.

```
git add Jenkinsfile
git commit -m "Add Jenkinsfile"
git push origin main
```

4 CREATING A PIPELINE JOB

1. Go to Jenkins Dashboard.
2. Click “New Item”.
3. Enter a name for your job.
4. Select “Pipeline”.
5. Click “OK”.
6. Under “Pipeline” section, select “Pipeline script from SCM”.
7. Choose your Git repository.
8. Set the branch (e.g., main).
9. Click “Save”.

5 RUNNING THE PIPELINE

1. Open your Pipeline job.
2. Click “Build Now”.
3. Jenkins will checkout the code.
4. It will read the Jenkinsfile.
5. It will execute the stages and steps.
6. Check the console output for the result.

6 READING STAGE VIEW

After the build starts, Jenkins shows the Stage View.

Build #1	Hello	Post Actions
Duration	✓	✓
2s	1s	<1s

- Each box represents a stage.
- Green = Success
- Red = Failure
- Blue = Running
- Click a stage to see details and logs.

7 WHAT HAPPENS BEHIND THE SCENES

GIT REPOSITORY
Contains Jenkinsfile

↓

JENKINS
Reads the Jenkinsfile

↓

PIPELINE EXECUTION
Runs stages and steps

↓

BUILD RESULT
Success / Failure

8 SUMMARY

- You created a Jenkinsfile.
- You saved it inside Git.
- You created a Pipeline job.
- You ran the Pipeline.
- You read the Stage View.

CONGRATULATIONS!
YOU JUST RAN YOUR FIRST PIPELINE!

TIP

Always keep your Jenkinsfile in source control. It makes your Pipelines repeatable and shareable.

You can add more stages like Build, Test, Deploy as your project grows.

Pipelines bring automation, consistency, and reliability to your software delivery.

12. BUILD, TEST, AND DEPLOY STAGES

1 CREATING MULTIPLE STAGES

Pipelines are divided into stages. Each stage represents a major step in your software delivery process. This makes your Pipeline organized, readable, and easy to maintain.

2 COMMON STAGES EXPLAINED

CHECKOUT	Gets the source code from Git repository.
BUILD	Compiles the code and creates the application.
TEST	Runs automated tests to verify the application.
DEPLOY	Deploys the application to the target environment.

3 WHY PIPELINES USE STAGES

- Breaks the process into logical steps.
- Improves readability and understanding.
- Makes it easy to see where failures happen.
- Enables parallel or conditional execution.
- Provides clear visibility in Stage View.
- Encourages best practices and consistency.

4 UNDERSTANDING EXECUTION ORDER

CHECKOUT → BUILD → TEST → DEPLOY

1. Stages run from top to bottom, in the order defined.
2. Each stage must complete before the next starts.
3. If a stage fails, the next stages are skipped.
4. The final result depends on all previous stages.

5 WHAT HAPPENS WHEN A STAGE FAILS?

If any stage fails:

- Jenkins marks the build as FAILURE.
- The remaining stages are skipped.
- You can see exactly which stage failed.
- You fix the issue and run the Pipeline again.

EXAMPLE

CHECKOUT → BUILD → TEST → DEPLOY

TEST fails, so DEPLOY is skipped.

6 SIMPLE CI/CD FLOW DIAGRAM

DEVELOPER	GIT	JENKINS	CHECKOUT	BUILD	TEST	DEPLOY
Writes code	Push code	Trigger Pipeline	Get Source	Compile Code	Run Tests	Release App

CONTINUOUS INTEGRATION (CI) — CONTINUOUS DELIVERY (CD)

7 EXAMPLE STRUCTURE

```
pipeline {
    agent any

    stages {
        stage('Build') {
            steps {
                echo 'Building application'
            }
        }

        stage('Test') {
            steps {
                echo 'Running tests'
            }
        }

        stage('Deploy') {
            steps {
                echo 'Deploying application'
            }
        }
    }
}
```

8 KEY TAKEAWAYS

- Use stages to organize your Pipeline into logical steps.
- Checkout → Build → Test → Deploy is the most common flow.
- Stages run in order and stop when a failure occurs.
- Stages make your Pipeline clear, maintainable, and professional.
- This is the foundation of any real CI/CD Pipeline.

BEST PRACTICE

- Keep stages small and focused.
- Use meaningful stage names.
- Fail fast and fix early.
- Review Stage View after every run.

JENKINS HANDBOOK FOR BEGINNERS

13. ENVIRONMENT VARIABLES AND PARAMETERS

1 WHAT ENVIRONMENT VARIABLES ARE

Environment variables store information that can be used during a build. They help make your Pipeline dynamic, reusable, and flexible.

2 JENKINS BUILT-IN VARIABLES

Jenkins provides many built-in variables you can use directly.

VARIABLE	DESCRIPTION	EXAMPLE
BUILD_NUMBER	The unique number of the current build.	42
JOB_NAME	The name of the job being built.	my-pipeline
BUILD_URL	The URL of the current build.	http://jenkins/job/my-pipeline/42/
WORKSPACE	The directory where the source code is checked out.	/var/lib/jenkins/workspace/my-pipeline
NODE_NAME	The name of the agent (node) running the build.	agent-01
GIT_COMMIT	The Git commit hash that was built.	a1b2c3d4

3 DEFINING ENVIRONMENT VARIABLES

You can define your own environment variables in your Jenkinsfile.

```
pipeline {
  agent any
  environment {
    APP_NAME = 'MyApp'
    APP_ENV  = 'DEV'
  }
}
```

EXAMPLE USE

```
echo "App: ${APP_NAME}"
echo "Environment: ${APP_ENV}"
```

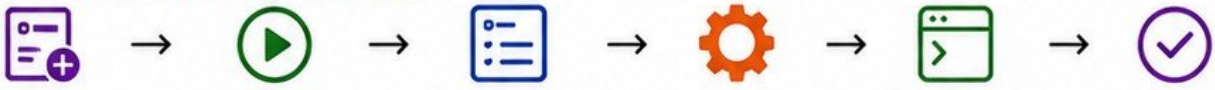
4 WHAT BUILD PARAMETERS ARE

Build parameters allow you to pass values from the user when triggering a build. This makes your Pipeline interactive and reusable for different situations.

5 TYPES OF BUILD PARAMETERS

STRING PARAMETER	• Accepts any text value. • Example: Environment name, version, tag, etc.
CHOICE PARAMETER	• Provides a list of predefined options. • Example: DEV, TEST, PROD
BOOLEAN PARAMETER	• True or False (checkbox). • Example: Run tests? Yes/No

6 RUNNING A PARAMETERIZED BUILD



1. **CONFIGURE PARAMETERS** — Add parameters in the job configuration.
2. **BUILD WITH PARAMETERS** — Click "Build with Parameters" on the job page.
3. **ENTER VALUES** — Fill in the parameter values and click "Build".
4. **JENKINS RUNS BUILD** — Jenkins uses the values in your Pipeline.
5. **VALUES AVAILABLE** — Use parameters in your steps and scripts.
6. **BUILD COMPLETES** — Result appears with the used parameter values.

7 EXAMPLE: PARAMETERS IN ACTION

PARAMETER	TYPE	EXAMPLE VALUE
ENVIRONMENT	CHOICE	DEV
VERSION	STRING	1.2.3
RUN_TESTS	BOOLEAN	true

USE IN JENKINSFILE

```
steps {
  echo "Deploying to ${params.ENVIRONMENT}"
  echo "Version: ${params.VERSION}"
  echo "Run Tests: ${params.RUN_TESTS}"
}
```

TIP

Parameters make your Pipeline flexible and reusable for different environments and scenarios without changing the code.

KEY TAKEAWAYS

- Environment variables store information for use during builds.
- Use Jenkins built-in variables to get useful information.
- Define your own variables for flexibility.
- Parameters let users provide values at build time.
- Pipelines with parameters are powerful and reusable.

JENKINS HANDBOOK FOR BEGINNERS

14. JENKINS PLUGINS

1 WHAT PLUGINS ARE

Plugins are add-ons that extend Jenkins with new features and integrations.

They allow Jenkins to work with many tools and technologies.

2 WHY JENKINS USES PLUGINS

- Jenkins is highly extensible.
- Plugins add features without changing Jenkins core.
- Plugins integrate Jenkins with other tools (Git, Docker, AWS, Kubernetes, Slack, etc.).
- Teams can customize Jenkins to fit their workflow.

3 PLUGIN MANAGER

All plugin management is done in the Plugin Manager.

Manage Jenkins → Manage Plugins

PLUGIN MANAGEMENT PROCESS

4 SEARCHING FOR PLUGINS

Go to the Available tab. Use the search box to find the plugin you need.

5 INSTALLING PLUGINS

Select the plugin checkbox and click "Install without restart" or "Download now and install after restart".

6 UPDATING PLUGINS

Go to the Updates tab. Select plugins with updates and click "Download now and install after restart".

7 RESTART REQUIREMENTS

Some plugins require Jenkins to restart to activate. Click "Restart Jenkins" after installation or update.

8 VERIFY INSTALLATION

After restart, check the Installed tab to confirm plugins are installed and active.

9 CONFIGURE IF NEEDED

Some plugins require global configuration before use. Check "Manage Jenkins" for plugin settings.

10 USE IN JOBS OR PIPELINES

Once installed and configured, you can use the plugin features in jobs or pipelines.

11 EXAMPLES OF USEFUL BEGINNER PLUGINS

PLUGIN	WHAT IT DOES	WHY IT'S USEFUL FOR BEGINNERS
GIT	Integrates Jenkins with Git.	Essential for checkout, branches, and source code management.
PIPELINE	Allows you to use Jenkins Pipelines.	Required to create Pipeline jobs and use Jenkinsfile.
BLUE OCEAN	Modern UI for Pipelines.	Makes Pipelines easier to view and understand.
CREDENTIALS	Manages credentials securely.	Needed for Git, Docker, cloud and other integrations.
WORKSPACE CLEANUP	Cleans up old build workspaces.	Keeps Jenkins clean and saves disk space.
BUILD TIMEOUT	Adds timeout to builds and stages.	Prevents stuck builds from running forever.

12 WHY BEGINNERS SHOULD AVOID INSTALLING UNNECESSARY PLUGINS

MORE PLUGINS = MORE COMPLEXITY
Too many plugins can make Jenkins difficult to manage.

PERFORMANCE IMPACT
Unnecessary plugins can slow down Jenkins startup and builds.

COMPATIBILITY ISSUES
Some plugins may not be compatible with your Jenkins version or other plugins.

SECURITY RISKS
Extra plugins increase the attack surface and can introduce vulnerabilities.

KEEP IT SIMPLE
Install only what you need. You can always install more later.

KEY TAKEAWAYS

- Plugins extend Jenkins capabilities.
- Use Plugin Manager to search, install, update, and manage.
- Restart Jenkins when required.
- Choose only necessary plugins for better performance and security.

15. JENKINS AGENTS AND NODES

1 WHY JENKINS USES AGENTS

Agents allow Jenkins to distribute work across multiple machines. This improves performance, reliability, and scalability.

2 CONTROLLER VERSUS AGENT

CONTROLLER

Manages Jenkins jobs, scheduling, and overall system.

AGENT

Runs the jobs assigned by the controller.

3 NODES

A node is any machine that Jenkins can use to run jobs.

The controller is also a node by default.

4 EXECUTORS

Executors are the number of concurrent jobs a node can run at the same time.

More executors = more parallel builds on that node.

5 LABELS

Labels help Jenkins identify and group nodes based on capabilities (e.g., linux, windows, docker).

Jobs can target nodes using these labels.

6 WORKSPACES

Each job gets a workspace on the agent where code is checked out and the build runs. Workspaces are separate on each agent.

RUNNING JOBS ON DIFFERENT AGENTS

7 AGENT ANY

agent any tells Jenkins to run the job on any available agent.

```
pipeline {
    agent any
    stages {
        stage('Build') {
            steps {
                echo 'Running on any available agent'
            }
        }
    }
}
```

- Jenkins picks the first available agent to run the job.
- Good for simple environments with similar agents.

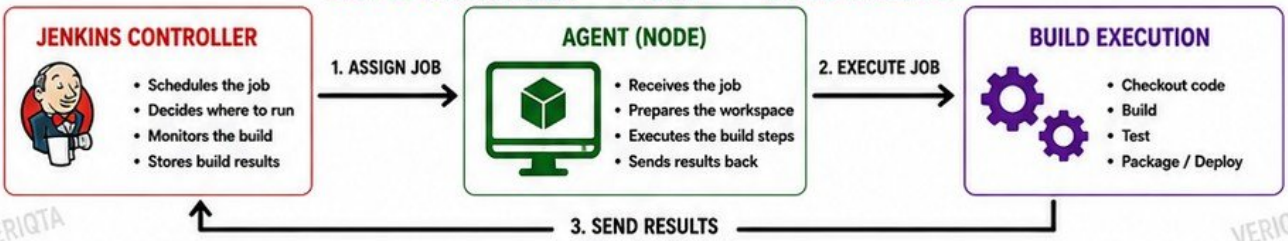
8 SELECTING AN AGENT USING LABELS

You can select a specific agent using a label.

```
pipeline {
    agent {
        label 'linux'
    }
    stages {
        stage('Build') {
            steps {
                echo 'Running on Linux agent'
            }
        }
    }
}
```

- Job runs only on agents that have the label "linux".
- Use labels to control where jobs run (e.g., linux, windows, docker).

SIMPLE CONTROLLER → AGENT → BUILD DIAGRAM



KEY TAKEAWAYS

- Agents help Jenkins scale and run jobs faster.
- Controller manages, agents execute.
- Use labels to target the right agents.
- Workspaces are isolated on each agent.

BEST PRACTICES

- Use multiple agents for better speed and reliability.
- Use labels to organize agents by OS, tools, or purpose.
- Avoid running heavy builds on the controller.
- Keep agents lightweight and dedicated to builds.

JENKINS HANDBOOK FOR BEGINNERS

16. ARTIFACTS AND BUILD RESULTS

1 WHAT BUILD ARTIFACTS ARE

Build artifacts are files or packages generated during the build process that are useful for later steps like testing, deployment, or distribution.

They are the outputs of your build.

2 EXAMPLES OF ARTIFACTS

- **JAR FILES** (Java applications)
- **WAR FILES** (Web applications)
- **ZIP PACKAGES** (Distributions, binaries, reports, documents)

3 ARCHIVING ARTIFACTS

Archiving saves important files from the build so they can be downloaded or used in later stages.

HOW TO ARCHIVE (FREESTYLE JOB)

Post-build Actions → Archive the artifacts

4 VIEWING ARCHIVED ARTIFACTS

1. Go to your job.
2. Click on a completed build.
3. Click "Artifacts" on the left menu.
4. You will see a list of archived files.
5. Click any file to download it.

5 TEST RESULTS

Test results show whether your code passed or failed automated tests.

- Jenkins can collect and display test results.
- Popular formats: JUnit (XML).
- Helps you identify failures quickly.

6 BUILD STATUS

- **SUCCESS** Everything completed without errors.
- **FAILURE** One or more steps failed.
- **UNSTABLE** Build completed but with issues (e.g., test failures).

Build status helps you understand the outcome at a glance.

7 KEEPING USEFUL OUTPUT FROM A BUILD

- Archive artifacts you need later.
- Store test reports and logs.
- Keep deployment packages.
- Preserve important configuration files.
- This helps in debugging, auditing, and future deployments.

8 EXAMPLE: ARCHIVING ARTIFACTS IN A PIPELINE

```
pipeline {
  agent any
  stages {
    stage('Build') {
      steps {
        echo 'Building application'
        // build commands
      }
    }
  }
  post {
    success {
      archiveArtifacts artifacts: 'target/*.jar', allowEmptyArchive: false
    }
  }
}
```

WHAT THIS DOES

- Archives all JAR files from the target folder.
- Makes them available after the build.
- You can download them from the Artifacts section.

PIPELINE STEPS FOR TEST RESULTS (EXAMPLE)

1. Add test step (e.g., using Maven, Gradle).
2. Publish test results using: junit 'target/surefire-reports/*.xml'
3. Jenkins will display test results in the job.

9 KEY TAKEAWAYS

- Artifacts are important outputs of your build.
- Archive artifacts to use them anytime.
- Test results help ensure code quality.
- Build status shows the health of your build.
- Keep useful files and logs for future use.

JENKINS HANDBOOK FOR BEGINNERS

17. POST ACTIONS AND NOTIFICATIONS

1 WHAT HAPPENS AFTER A PIPELINE FINISHES

After a Pipeline completes, Jenkins can run additional steps automatically. These are called Post Actions.

2 THE POST SECTION

The post section in a Pipeline lets you define actions to run based on the result of the build.

It helps with notifications, cleanup, and handling success or failure.

3 POST CONDITIONS EXPLAINED

CONDITION	WHEN IT RUNS
ALWAYS	Runs no matter what (success, failure, or unstable).
SUCCESS	Runs only when the Pipeline succeeds.
FAILURE	Runs only when the Pipeline fails.
CLEANUP	Used to clean workspace or temporary files.

4 RUNNING ACTIONS AFTER BUILDS

- Post actions help automate important tasks after every build.
- Examples: send notifications, clean files, archive reports, update systems.
- They reduce manual work and improve consistency.

5 BASIC NOTIFICATION CONCEPT

- Notifications keep your team informed.
- You can notify on success, failure, or always.
- Common methods: Email, Slack, Microsoft Teams, Discord, Telegram.
- Many notifications are done using plugins.

6 CLEANING TEMPORARY FILES

- Always clean up files after a build to save disk space.
- You can delete workspaces, temporary files, and caches.
- Use the `deleteDir()` step or workspace cleanup plugins.

7 SIMPLE SUCCESS AND FAILURE HANDLING

- Take different actions based on the build result.
- Send a success notification when everything is good.
- Send an alert when something fails.
- This helps your team react quickly.

8 EXAMPLE: POST ACTIONS IN A PIPELINE

```groovy
pipeline {
    agent any
    stages {
        stage('Build') {
            steps {
                echo 'Building application'
            }
        }
        stage('Test') {
            steps {
                echo 'Running tests'
            }
        }
        stage('Deploy') {
            steps {
                echo 'Deploying application'
            }
        }
    }

    post {
        always {
            echo 'This runs no matter what'
            cleanWs()
        }
        success {
            echo 'Build succeeded! Sending success notification...'
        }
        failure {
            echo 'Build failed! Sending failure notification...'
        }
        cleanup {
            echo 'Cleaning up temporary files...'
            deleteDir()
        }
    }
}
```

HOW THIS WORKS

ALWAYS	Runs every time the Pipeline finishes.
SUCCESS	Runs only when the build is successful.
FAILURE	Runs only when the build fails.
CLEANUP	Used to clean workspace or temporary files.

9 EXAMPLE USE CASES

- Send email when build fails.
- Notify Slack on success.
- Archive logs after every build.
- Clean workspace to save disk space.
- Upload test reports.

KEY TAKEAWAYS

- Post actions run after the Pipeline finishes.
- Use post to handle notifications, cleanup, and more.
- Use the right condition for the right situation.
- Always clean up temporary files and workspaces.
- Notifications help your team respond faster.

18. TROUBLESHOOTING JENKINS FOR BEGINNERS

1 JOB DOES NOT START

- Check if Jenkins is online.
- Verify the job is enabled.
- Check for scheduling issues.
- Look for quiet period delay.
- Check node (agent) availability.

2 GIT REPOSITORY CANNOT BE CLONED

- Verify repository URL.
- Check network connectivity.
- Verify credentials.
- Check SSH keys.
- Ensure the branch exists.

3 CREDENTIALS FAIL

- Verify the correct credentials are used.
- Check username/password or token.
- Ensure credentials have access to the resource.
- Check if credentials expired.

4 COMMAND NOT FOUND

- The tool may not be installed on the agent.
- Check PATH variable.
- Use full path to the command.
- Verify tools are installed on the correct node.

5 PERMISSION DENIED

- Check file or directory permissions.
- Verify user permissions on the agent.
- Check Jenkins user permissions.
- For SSH, check key permissions.

6 PLUGIN PROBLEMS

- Plugin may be outdated or incompatible.
- Check plugin version.
- Update the plugin.
- Restart Jenkins after installing/updating.
- Check plugin logs.

7 PIPELINE SYNTAX ERRORS

- Check Jenkinsfile syntax.
- Use the Pipeline Syntax Checker.
- Look for missing braces, quotes, or commas.
- Read the error message carefully.

8 WORKSPACE PROBLEMS

- Workspace may be locked.
- Try "Clean Workspace" before build.
- Disk full can cause failures.
- Check file permissions in the workspace.

9 BUILD FAILS UNEXPECTEDLY

- Read the full console output.
- Check recent changes.
- Verify environment and dependencies.
- Check test failures.
- Re-run with more logging if needed.

10 USING CONSOLE OUTPUT TO FIND THE ACTUAL ERROR

The Console Output is your best friend. Always read from bottom to top to find the real error.

- Look for lines with ERROR, Exception, Failed, or stack trace.
- Ignore repeated lines.
- The first error is usually the root cause.
- Copy the error message and search for solutions.

EXAMPLE CONSOLE OUTPUT (LOOK FOR THE REAL ERROR)

```
[INFO] Building project...
[INFO] Running tests
[INFO] Tests run: 20, Failures: 1
[ERROR] there was 1 failure:
[ERROR] testLogin(com.example.Test) Time elapsed: 0.123 s <<< FAILURE!
  java.lang.NullPointerException
     at com.example.Test.testLogin(Test.java:45)
Finished: FAILURE
```

11 BASIC TROUBLESHOOTING ORDER

1. **READ THE ERROR CAREFULLY**
   Check the last part of the Console Output.
2. **IDENTIFY THE PROBLEM TYPE**
   Is it network, permission, syntax, missing tool, credential, or plugin?
3. **CHECK CONFIGURATION**
   Review job configuration, environment, tools, and settings.
4. **VERIFY ENVIRONMENT**
   Check agent (node), tools, credentials, and workspace.
5. **FIX AND RE-RUN**
   Apply the fix and run the build again. Test one change at a time.

KEY TAKEAWAYS

- Always check the Console Output first.
- Understand the type of error.
- Verify configuration, tools, and credentials.
- Use logs and error messages to guide your fix.
- Fix one thing at a time and retest.

JENKINS HANDBOOK FOR BEGINNERS

19. BEGINNER CI/CD PROJECT

BUILD A COMPLETE BEGINNER JENKINS WORKFLOW

1 CREATE A SIMPLE GITHUB REPOSITORY

- Create a new repository on GitHub.
- Name it e.g., jenkins-ci-cd-demo.

2 ADD APPLICATION FILES

- Add a simple application
- (e.g., Java Maven or Python).
- Commit and push the files.

3 ADD A JENKINSFILE

- Create a Jenkinsfile in the root of your repository.
- Commit and push it.

4 CONNECT JENKINS TO THE REPOSITORY

- Create a new Pipeline job in Jenkins.
- Connect it to your GitHub repository.

5 CONFIGURE AUTOMATIC TRIGGERING

- Enable GitHub hook trigger for GITScm polling or webhook.
- This will trigger the build automatically on push.

6 CHECKOUT SOURCE CODE

- Jenkins will checkout the latest code from GitHub.
- Workspace is prepared.

7 BUILD THE APPLICATION

- Compile or package the application using Maven, Gradle, or other tools.
- Build the code successfully.

8 RUN A SIMPLE TEST

- Run unit tests or basic tests.
- Verify that the code works as expected.

9 CREATE AN ARTIFACT

- Package the application (e.g., JAR, WAR, ZIP).
- This becomes your build artifact.

10 ARCHIVE THE ARTIFACT

- Archive the artifact in Jenkins.
- It will be available for download later.

11 ADD SUCCESS AND FAILURE ACTIONS

- On success: Send a success notification.
- On failure: Send a failure notification and capture logs.

EXAMPLE JENKINSFILE (SIMPLE CI PIPELINE)

```
pipeline {
  agent any

  stages {
    stage('Checkout') {
       steps {
          checkout scm
       }
    }
    stage('Build') {
       steps {
        sh 'mvn clean package'
       }
    }
    stage('Test') {
       steps {
          sh 'mvn test'
       }
    }
  }
  post {
     success { echo 'Build succeeded! Notification sent.' }
     failure { echo 'Build failed! Check logs and fix.') }
  }
```

WHAT HAPPENS WHEN YOU PUSH A CHANGE

1. You push a change to GitHub.
2. GitHub triggers Jenkins automatically.
3. Jenkins runs the Pipeline.
4. Build, test, and artifact are created.
5. You get a success or failure notification.
6. Artifacts are archived and available.

FINAL FLOW

DEVELOPER → GITHUB → JENKINS → CHECKOUT → BUILD → TEST → ARTIFACT

KEY TAKEAWAYS

- Jenkins automates your software delivery process.
- Every push can trigger a complete CI/CD workflow.
- Artifacts are your deliverables.
- Notifications keep your team informed.
- Small steps build real CI/CD experience.

JENKINS HANDBOOK FOR BEGINNERS

20. JENKINS BEGINNER REFERENCE AND NEXT STEPS

1 JENKINS TERMINOLOGY RECAP

CONTROLLER	The main Jenkins server that manages everything.
AGENT	A machine that runs jobs for Jenkins.
NODE	A Jenkins agent (also called a slave in older terms).
EXECUTOR	A worker process on a node that runs a build.
JOB	A task configured in Jenkins to perform work.
BUILD	A single execution of a job.
WORKSPACE	A directory on the agent where files are checked out.
PIPELINE	A defined flow of stages and steps in a Jenkinsfile.
STAGE	A logical section of a pipeline (e.g., Build, Test, Deploy).
STEP	A single execution within a stage.
JENKINSFILE	A text file that defines a Pipeline as code.
ARTIFACT	The output files generated by a build.
CREDENTIAL	Secure information used to access resources.
PLUGIN	An add-on that extends Jenkins functionality.
WEBHOOK	An HTTP callback used to trigger Jenkins.

2 ESSENTIAL JENKINSFILE SYNTAX

```
pipeline {
  agent any
  environment {
    APP_NAME = 'my-app'
  }
  parameters {
    string(name: 'ENV', defaultValue: 'dev')
  }
  stages {
    stage('Build') {
      steps {
        echo 'Building application'
      }
    }
    stage('Test') {
      steps {
        echo 'Running tests'
      }
    }
    stage('Deploy') {
      steps {
        echo "Deploying to ${params.ENV}"
      }
    }
  }
  post {
    success { echo 'Build succeeded!' }
    failure { echo 'Build failed!' }
    always { cleanWs() }
  }
}
```

KEY POINTS

- ✔ **pipeline:** Defines a declarative Pipeline.
- ✔ **agent:** Chooses the agent to run on.
- ✔ **environment:** Defines variables.
- ✔ **parameters:** User inputs for the build.
- ✔ **stages:** Logical sections.
- ✔ **steps:** Actions inside a stage.
- ✔ **post:** Actions after the Pipeline completes.

3 ESSENTIAL BEGINNER TROUBLESHOOTING CHECKLIST

- ☐ **CHECK CONSOLE OUTPUT FIRST** — Read from top to bottom.
- ☐ **VERIFY JOB CONFIGURATION** — Check triggers, paths, and settings.
- ☐ **CHECK AGENT AVAILABILITY** — Is the agent online and has executors?
- ☐ **CHECK REPOSITORY ACCESS** — Can Jenkins clone the repo?
- ☐ **VERIFY CREDENTIALS** — Are credentials valid and have access?
- ☐ **CHECK PERMISSIONS** — File permissions, user permissions, and tokens.
- ☐ **CHECK TOOLS AND PLUGINS** — Is the required tool/plugin installed?
- ☐ **RE-PRODUCE LOCALLY** — Run the same command on the agent.
- ☐ **CHECK DISK SPACE AND MEMORY** — Low resources can cause failures.
- ☐ **RE-RUN WITH MORE LOGGING** — Add -x, --info, or debug flags.

4 SKILLS YOU NOW UNDERSTAND

- **JENKINS BASICS** — What Jenkins is and how it works.
- **PIPELINES** — Creating and running Pipelines with Jenkinsfile.
- **STAGES AND STEPS** — Structuring builds using stages and steps.
- **GIT INTEGRATION** — Connecting Jenkins with GitHub.
- **BUILD PROCESS** — Checkout, build, test, and archive.
- **TROUBLESHOOTING** — Using Console Output to find issues.
- **ARTIFACT MANAGEMENT** — Archiving and using build outputs.
- **POST ACTIONS** — Notifications and cleanup actions.
- **PROJECT CREATION** — Building a complete CI/CD workflow.

5 WHAT TO LEARN NEXT

- **DOCKER** — Containerize applications and run builds in Docker.
- **MAVEN OR GRADLE** — Build automation tools for Java projects.
- **TESTING** — Unit testing, integration testing, and test reports.
- **DEPLOYMENT PIPELINES** — Deploy to environments like staging and production.
- **SHARED LIBRARIES** — Reuse Pipeline code using shared libraries.
- **JENKINS ADMINISTRATION** — Manage users, security, nodes, backups, and settings.
- **MONITORING AND LOGGING** — Use plugins for monitoring, alerts, and dashboards.
- **ADVANCED TOPICS** — Parallel stages, matrix builds, approvals, and more.

6 YOUR JOURNEY CONTINUES – KEEP PRACTICING!

LEARN Understand concepts. → **PRACTICE** Build small projects. → **BUILD** Create real workflows. → **IMPROVE** Optimize and troubleshoot. → **MASTER** Become a Jenkins Expert!

REMEMBER: PRACTICE + CONSISTENCY = JENKINS SKILLS!